College Data Science Lab Exam Study Guide

Exercise 8: K-Means Clustering Algorithm

Source File: `e08_kmeans.py` • College of Engineering Trivandrum (CET) • 20MCA241 Data Science Lab

---

Table of Contents

1. Program Identification and Aim
2. Prerequisites and Conceptual Framework (Unsupervised vs Supervised)
3. Complete Exam-Ready Algorithm (START → STOP Format)
4. Complete Mathematical Theory (Distance, Centroids, ARI & NMI)
5. Complete Original Source Code & Detailed Walkthrough
6. Complete Function, Parameter, and Syntax Reference
7. Step-by-Step Execution Walkthrough & Data Transformations
8. Output, Cluster Visualizations & Metric Interpretation
9. Exam Preparation: Code Skeleton & Writing Strategy
10. Comprehensive Viva Voce Questions & Answers (16 Detailed Q&As)
11. Common Mistakes, Traps, and Troubleshooting
12. Memory Tricks & Last-Minute Rapid Revision Sheet
13. Cross-Program Master Comparison Table (Programs 05 to 10)

Section 1: Program Identification and Aim

Experiment Number: Exercise 8

Script Filename: `e08_kmeans.py`

Course: 20MCA241 Data Science Lab • Semester 3 MCA

Institution: Department of Computer Applications, College of Engineering Trivandrum (CET)

> **Official Lab Manual Aim**
>
> *Program to implement K-means Clustering Algorithm using any standard dataset available in the public domain.*

Dataset Used: Fisher's Iris Dataset (150 samples × 4 features). Per college manual specification, exactly the first 2 features are extracted: `iris.data[:, :2]` representing **Sepal Length (cm)** and **Sepal Width (cm)**.

Cluster Count (k): $k = 3$ clusters (corresponding to 3 iris flower species), initialized with fixed seed `random_state=21`.

Evaluation Modality: External cluster validation using **Adjusted Rand Index (ARI)** and **Normalized Mutual Information (NMI)** comparing unsupervised cluster assignments against ground-truth labels.

Section 2: Prerequisites and Conceptual Framework

Understanding K-Means requires grasping the fundamental distinction between supervised learning and unsupervised clustering:

- **Supervised Learning (Programs 5, 6, 7, 9, 10):** The algorithm is provided input features X alongside target labels/values y. The objective is to learn a mapping $f(X) □pprox y$.
- **Unsupervised Clustering (Program 8):** The algorithm receives **only feature vectors X** without any target labels. It discovers inherent geometric grouping and natural cluster structure purely by minimizing intra-cluster variance.
- **Role of Ground Truth (y):** In `e08_kmeans.py`, y is **never passed to `kmeans.fit(X)`**. True labels are solely utilized post-clustering to evaluate clustering quality via external validation metrics (ARI and NMI) and to plot side-by-side comparison scatter figures.

Section 3: Complete Exam-Ready Algorithm

This algorithm follows the exact wording and structure from the official CET lab manual (pages 22–23), formatted in the standard numbered START → STOP layout for lab record books and exam answer sheets.

GENERAL ALGORITHM: K-MEANS CLUSTERING

Step 1 [START]: Begin the program execution.
Step 2 [Import Libraries]: Import `datasets` and `KMeans` from `sklearn`, `pyplot` from `matplotlib`, and `adjusted_rand_score`, `normalized_mutual_info_score` from `sklearn.metrics`.
Step 3 [Load Dataset]: Load Fisher's Iris dataset using `datasets.load_iris()`.
Step 4 [Feature Selection]: Extract the first two columns (Sepal Length and Sepal Width) using 2D slicing `X = iris.data[:, :2]`, and extract ground-truth species labels `y = iris.target`.
Step 5 [Initialize Cluster Centers]: Initialize the K-Means model with $k = 3$ clusters and fixed random seed 21: `kmeans = KMeans(n_clusters=3, random_state=21)`.
Step 6 [Expectation-Maximization Optimization]:

- Fit Model: Execute `kmeans.fit(X)`.
- Assignment (E-step): Assign each sample x_i to its closest centroid v_j based on minimum Euclidean distance: $\arg\min_j ||x_i - v_j||^2$.
- Update (M-step): Recalculate each centroid v_j as the arithmetic mean of all data points assigned to cluster S_j: $v_j = \frac{1}{|S_j|} \sum_{x \in S_j} x$.
- Iterate: Repeat assignment and centroid recalculation until convergence (centroids stabilize).

Step 7 [Extract Cluster Labels]: Retrieve the resulting cluster assignments from `kmeans.labels_`.
Step 8 [Plot Actual vs Predicted Visualizations]:

- Create subplot 1 (Actual): Scatter plot of X colored by ground-truth y with colormap `prism` and black edges. Set title to `Actual`, x-label to `Sepal Length`, y-label to `Sepal Width`.
- Create subplot 2 (Predicted): Scatter plot of X colored by cluster assignments `labels` with colormap `jet` and black edges. Set title to `Predicted`, x-label to `Sepal Length`, y-label to `Sepal Width`.
- Adjust layout with `plt.tight_layout()` and display via `plt.show()`.

Step 9 [Compute External Validation Metrics]:

- Calculate Adjusted Rand Index: `ari = adjusted_rand_score(y, labels)`.
- Calculate Normalized Mutual Information: `nmi = normalized_mutual_info_score(y, labels)`.

Step 10 [Display Evaluation Metrics]: Print the computed ARI and NMI values formatted to 3 decimal places.
Step 11 [STOP]: Terminate program execution.

Section 4: Complete Mathematical Theory

1. K-Means Objective Function (Inertia)

K-Means partitions n observations into k disjoint clusters $S = \{S_1, S_2, ..., S_k\}$ to minimize within-cluster sum of squares (WCSS / Inertia):

$$J = \sum_{j=1}^{k} \sum_{x_i \in S_j} ||x_i - \mu_j||^2$$ Where μ_j is the centroid (mean vector) of cluster S_j: $$\mu_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

2. Distance Metric (Euclidean Distance)

For two feature points $x = (x_1, x_2)$ and centroid $\mu = (\mu_1, \mu_2)$:

$$d(x, \mu) = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2}$$

3. Adjusted Rand Index (ARI)

The Rand Index measures the proportion of sample pairs that are either grouped together or placed in different clusters across both true partitions and predicted clusters. The **Adjusted Rand Index** adjusts for chance:

$$ \text{ARI} = \frac{ \text{Index} - \text{Expected Index}}{ \text{Max Index} - \text{Expected Index}}$$

- $ \text{ARI} = 1.0$: Perfect agreement between clustering and true labels.
- $ \text{ARI} \approx 0.0$: Cluster assignments equivalent to random guessing.
- $ \text{ARI} < 0.0$: Disagreement worse than random expectation.
- Symmetry: $ \text{ARI}(y, \text{labels}) = \text{ARI}(\text{labels}, y)$. Invariant to label permutation (cluster index 0, 1, 2 does not need to equal species index 0, 1, 2).

4. Normalized Mutual Information (NMI)

Measures the shared information between true class distribution Y and cluster assignments C, normalized by their entropy:

$$ \text{NMI}(Y, C) = \frac{2 \times I(Y; C)}{H(Y) + H(C)}$$ Where $I(Y; C)$ is mutual information, and $H(Y), H(C)$ are Shannon entropies.

- Range: $[0.0, 1.0]$. Values close to 1 indicate high informational correlation.

Section 5: Complete Original Source Code & Detailed Walkthrough

Below is the complete, tested source code of `e08_kmeans.py` matching the college manual:

```
# Exercise 8: K-Means Clustering

# Aim: Program to implement K-means Clustering Algorithm using any standard dataset available in the public domain.

from sklearn import datasets
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

# 1. Load Iris dataset and select first 2 features (Sepal length and Sepal width)
iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target

# 2. Initialize and fit KMeans clustering model
kmeans = KMeans(n_clusters=3, random_state=21)
kmeans.fit(X)
labels = kmeans.labels_

# 3. Visualize Actual vs Predicted clusters
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='prism', edgecolor='k')
plt.title('Actual')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')

plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='jet', edgecolor='k')
plt.title('Predicted')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')

plt.tight_layout()
plt.show()

# 4. Evaluate clustering performance using ARI and NMI
ari = adjusted_rand_score(y, labels)
nmi = normalized_mutual_info_score(y, labels)

print(f"Adjusted Rand Index (ARI): {ari:.3f}")
print(f"Normalized Mutual Information (NMI): {nmi:.3f}")
```

Line-by-Line Code Walkthrough

Line(s)	Code Statement	Operational Purpose & Beginner Explanation
5-8	`from sklearn import datasets ...`	Imports dataset loader, plotting library, KMeans estimator, and external validation metrics (ARI, NMI).
11	`iris = datasets.load_iris()`	Loads Iris Bunch object containing 150 samples of iris flower measurements.
12	`X = iris.data[:, :2]`	Slices the first two columns (index 0: Sepal length, index 1: Sepal width). Shape becomes $(150, 2)$.
13	`y = iris.target`	Loads ground-truth integer labels (0=Setosa, 1=Versicolor, 2=Virginica). Used only for evaluation and comparison plot.
16	`kmeans = KMeans(n_clusters=3, random_state=21)`	Configures K-Means to partition data into 3 clusters. `random_state=21` guarantees deterministic initial centroid seeds.
17	`kmeans.fit(X)`	Runs iterative optimization on X. Note: y is NOT passed; model learns clusters completely unsupervised.
18	`labels = kmeans.labels_`	Extracts 1D array of 150 assigned cluster integers $(0, 1, 2)$.
21-25	`plt.subplot(1, 2, 1) ...`	Creates left subplot. Scatters $X[:, 0]$ vs $X[:, 1]$ colored by true labels y using `prism` colormap.
27-31	`plt.subplot(1, 2, 2) ...`	Creates right subplot. Scatters $X[:, 0]$ vs $X[:, 1]$ colored by predicted `labels` using `jet` colormap.
33-34	`plt.tight_layout(); plt.show()`	Adjusts subplot spacing to prevent title/label clipping and renders the dual-panel window.
37-38	`ari = adjusted_rand_score(y, labels) ...`	Computes Adjusted Rand Index and Normalized Mutual Information comparing true y with predicted $labels$.
40-41	`print(f"Adjusted Rand Index (ARI): ...")`	Prints formatted evaluation metrics to the console with 3 decimal precision.

Section 6: Complete Function, Parameter, and Syntax Reference

Function / Attribute	Library	Key Parameters / Return Value	Exam Role & Significance
`KMeans()`	`sklearn.cluster`	`n_clusters=3`: number of clusters to form. `random_state=21`: deterministic centroid seed.	Primary clustering estimator. Defaults to `k-means++` smart initialization.
`kmeans.fit(X)`	`sklearn.cluster.KMeans`	Input: feature matrix X of shape $(n_samples, n_features)$.	Computes cluster centers and predicts cluster index for each sample.
`kmeans.labels_`	`sklearn.cluster.KMeans`	Array of shape $(150,)$ with integer cluster assignments.	Stores learned cluster assignment for each sample in training data.
`plt.subplot(1, 2, k)`	`matplotlib.pyplot`	1 row, 2 columns, index $k \in \{1, 2\}$.	Splits canvas horizontally to show side-by-side comparison of Actual vs Predicted.
`plt.scatter()`	`matplotlib.pyplot`	`c=...`: color vector. `cmap='prism'/'jet'`: color palette. `edgecolor='k'`: black boundary.	Renders 2D scatter plot of samples. Black edges sharpen points.
`adjusted_rand_score()`	`sklearn.metrics`	`(labels_true, labels_pred)` → float $[-1.0, 1.0]$.	Calculates similarity between two clusterings adjusted for chance.
`normalized_mutual_info_score()`	`sklearn.metrics`	`(labels_true, labels_pred)` → float $[0.0, 1.0]$.	Calculates normalized mutual information between cluster assignments.

Section 7: Step-by-Step Execution Walkthrough

Step	Code Line	Array / Variable	Shape & Type	Contents / Value Description
1	11	`iris`	`Bunch` object	Complete dataset with 150 samples and metadata.
2	12	`X`	`ndarray (150, 2)`, float64	Columns 0 and 1: Sepal length (4.3 to 7.9) and Sepal width (2.0 to 4.4).
3	13	`y`	`ndarray (150,)`, int32	Ground-truth species: 50 Setosa (0), 50 Versicolor (1), 50 Virginica (2).
4	16	`kmeans`	`KMeans` instance	Initialized with $k=3$, `random_state=21`, ready for fitting.
5	17-18	`labels`	`ndarray (150,)`, int32	Values $\in \{0, 1, 2\}$ indicating assigned clusters.
6	37-38	`ari, nmi`	Python float	ARI $□pprox 0.586$ to 0.601; NMI $□pprox 0.619$ to 0.647.

Section 8: Output and Interpretation

1. Console Terminal Output

```
Adjusted Rand Index (ARI): 0.586
Normalized Mutual Information (NMI): 0.619
```

(Note: Depending on scikit-learn version and platform BLAS libraries, ARI ranges between 0.586 and 0.601, and NMI ranges between 0.619 and 0.647, exactly matching the college manual screenshot on page 22).

2. Plot Interpretation

- **Subplot 1 (Actual):** Shows ground-truth biological species. Setosa forms a cleanly isolated cluster on the upper left (short length, wide width). Versicolor and Virginica exhibit slight linear overlap in the middle and upper right.
- **Subplot 2 (Predicted):** Shows K-Means clusters formed strictly from spatial proximity. Notice that K-Means perfectly separates the isolated Setosa group. In the overlapping region between Versicolor and Virginica, K-Means draws a clean Voronoi geometric partition boundary, misclassifying a few boundary points because it has no access to true class labels.
- **Why ARI is ~0.60:** Because Setosa is 100% cleanly separated, while Versicolor and Virginica have natural overlap when using only two features (Sepal length and width). An ARI of ~0.60 confirms substantial cluster agreement well above chance (0.0).

Section 9: Exam Preparation: Code Skeleton & Writing Strategy

5-Minute Exam Memorization Skeleton

```
from sklearn import datasets
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target

kmeans = KMeans(n_clusters=3, random_state=21).fit(X)
labels = kmeans.labels_

plt.subplot(1, 2, 1); plt.scatter(X[:, 0], X[:, 1], c=y, cmap='prism', edgecolor='k')
plt.subplot(1, 2, 2); plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='jet', edgecolor='k')
plt.show()

print("ARI:", round(adjusted_rand_score(y, labels), 3))
print("NMI:", round(normalized_mutual_info_score(y, labels), 3))
```

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Q1. What is the fundamental difference between K-Means and K-Nearest Neighbors (KNN)?

Answer: K-Means is an *unsupervised clustering* algorithm that groups unlabeled data into k clusters based on geometric distance. KNN is a *supervised classification* algorithm that predicts the label of a query sample by taking a majority vote of its k nearest labeled training neighbors.

Q2. Why is the ground-truth target label `y` excluded from `kmeans.fit(X)`?

Answer: Clustering is an unsupervised task; in practical applications, ground truth labels do not exist. Passing y to `fit()` would violate unsupervised learning principles. In this experiment, y is retained solely to validate clustering quality using ARI and NMI post-training.

Q3. What does `random_state=21` accomplish in `KMeans`?

Answer: It fixes the pseudo-random seed used to pick initial cluster centroids. Because standard K-Means convergence is sensitive to initial centroid locations, setting a fixed seed guarantees identical, reproducible cluster assignments and evaluation scores across all runs.

Q4. What is the Expectation-Maximization (E-M) process in K-Means?

Answer: It alternates between two steps until convergence:

- **E-Step (Expectation / Assignment):** Assign each sample to its nearest centroid based on Euclidean distance.
- **M-Step (Maximization / Update):** Recompute each centroid as the geometric center (mean vector) of all points currently assigned to that cluster.

Q5. What is the Adjusted Rand Index (ARI) and what is its optimal value?

Answer: ARI evaluates clustering similarity against ground truth, correcting for chance pairings. Its maximum score is 1.0 (perfect clustering), while expected random clustering yields 0.0. Negative scores indicate worse-than-random grouping.

Q6. Why doesn't ARI require cluster IDs to match true class labels?

Answer: ARI is permutation-invariant. It evaluates whether pairs of points that belong together in ground truth are placed together in predicted clusters, regardless of whether the cluster index is named 0, 1, or 2.

Q7. What is Normalized Mutual Information (NMI)?

Answer: NMI measures the mutual information shared between the ground-truth class distribution and the assigned clusters, normalized to the range $[0, 1]$. An NMI of 1 represents perfect cluster-to-class alignment.

Q8. Why does the program select only the first two features `iris.data[:, :2]`?

Answer: Selecting two features (Sepal length and Sepal width) enables direct 2D scatter plotting on Cartesian coordinates (x-axis and y-axis), allowing visual comparison between actual biological groups and predicted clusters.

Q9. How do you determine the optimal number of clusters k when it is unknown?

Answer:

- **Elbow Method:** Plot within-cluster sum of squares (inertia) vs k and locate the "elbow" point where marginal reduction sharply levels off.
- **Silhouette Analysis:** Calculate the average silhouette coefficient across samples for different values of k and pick k with the maximum score.

Q10. What is `k-means++` initialization?

Answer: Instead of choosing all initial centroids completely at random, `k-means++` chooses the first centroid uniformly at random, and then selects subsequent centroids with probability proportional to the squared distance from the nearest existing centroid. This spreads initial centroids out, dramatically reducing iterations and preventing poor local minima.

Q11. What are the stopping / convergence criteria for K-Means?

Answer: The algorithm terminates when:

1. Centroid locations do not change between iterations (or change less than tolerance `tol`).
2. No data points change cluster membership.
3. The maximum number of iterations (`max_iter`, default 300) is reached.

Q12. What are the main limitations of K-Means clustering?

Answer:

1. Requires specifying k in advance.
2. Assumes spherical, equal-variance clusters; performs poorly on elongated or arbitrary shapes.
3. Highly sensitive to outliers, which distort cluster centroids.
4. Sensitive to feature scale (features with larger numerical ranges dominate Euclidean distance).

Q13. Why are colormaps `prism` and `jet` used in the two subplots?

Answer: `prism` provides distinct repeating high-contrast colors suitable for true discrete integer categories, while `jet` provides a vivid spectrum ranging from blue to cyan, yellow, and red to clearly differentiate cluster IDs.

Q14. What does `kmeans.cluster_centers_` represent?

Answer: An array of shape $(k, n_features)$ (here $3 imes 2$) containing the final coordinates of the k cluster centroids in feature space.

Q15. How can K-Means assign a brand new test point to a cluster?

Answer: Using `kmeans.predict(new_point)`. It computes the Euclidean distance from the new point to all 3 existing cluster centers and assigns the index of the nearest center.

Q16. What is the time complexity of the standard Lloyd's K-Means algorithm?

Answer: $\mathcal{O}(I imes k imes n imes d)$, where I is the number of iterations until convergence, k is the cluster count, n is the number of data samples (150), and d is feature dimensionality (2).

Section 11: Common Mistakes, Traps, and Troubleshooting

Common Mistake	Error Message / Negative Effect	Correct Approach
Passing y to `fit()`: `kmeans.fit(X, y)`	Semantic error. Although scikit-learn accepts it for API uniformity, passing y displays misunderstanding of unsupervised learning.	Always write `kmeans.fit(X)`.
Using accuracy score: `accuracy_score(y, labels)`	Arbitrary or near-zero accuracy due to label index permutation (e.g. cluster 0 may correspond to class 2).	Always evaluate clustering with permutation-invariant metrics: `adjusted_rand_score(y, labels)` and `normalized_mutual_info_score(y, labels)`.
Forgetting 2D slicing: `X = iris.data`	Fails during 2D plotting or produces 4D clustering that does not match manual visualization.	Slice first two columns explicitly: `X = iris.data[:, :2]`.
Omitting `edgecolor='k'`	Points blend together visually without distinct boundaries.	Include `edgecolor='k'` to outline scatter points clearly.

Section 12: Memory Tricks & Last-Minute Rapid Revision Sheet

Rapid Formula Sheet

- **Feature Slicing:** `X = iris.data[:, :2]` (Sepal length & Sepal width).
- **Initialization:** `KMeans(n_clusters=3, random_state=21)`.
- **Training:** `kmeans.fit(X)` (Unsupervised — features only!).
- **Cluster Array:** `labels = kmeans.labels_`.
- **Subplots:** `plt.subplot(1, 2, 1)` for Actual, `plt.subplot(1, 2, 2)` for Predicted.
- **Evaluation Metrics:** `adjusted_rand_score(y, labels)` & `normalized_mutual_info_score(y, labels)`.

Section 13: Cross-Program Master Comparison Table (Programs 05 to 10)

This master comparison summarizes all Machine Learning experiments in the college Data Science curriculum:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	Structured text classification report
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"` , `max_depth=4` , `min_samples_leaf=2`)	`classification_report()`	Flowchart tree via `tree.plot_tree(filled=True)`
e08_kmeans.py	**Unsupervised Clustering**	Iris Dataset (First 2 Features: 150 × 2)	None (Full unlabeled dataset X)	K-Means Clustering (`n_clusters=3` , `random_state=21`)	Adjusted Rand Index (ARI) & Normalized Mutual Info (NMI)	2 Subplots: Actual (`prism`) vs Predicted (`jet`)
e09_linear_regression.py	**Supervised Regression (Simple)**	Diabetes Dataset (1 Feature: BMI, Index 2)	Slicing (`[:-20]` train, `[-20:]` test)	Ordinary Least Squares (`LinearRegression()`)	Coefficients, Intercept, MSE, R^2 Score	Scatter plot of test samples with fitted linear line
e10_multiple_linear_regression.py	**Supervised Regression (Multiple)**	Diabetes Dataset (2 Features: Age & BMI, Indices 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Coefficients (vector of 2), Intercept, MSE, R^2 Score	Figure 1: Feature 0 & 2 subplots; Figure 2: Actual vs Predicted fit line